

Experiment 1:

Develop a Python program using OpenAI Gym (FrozenLake) where the agent reaches the goal while avoiding holes. Explain the state space, action space, and reward function.

Program:

```
import gymnasium as gym
import random

env = gym.make("FrozenLake-v1", is_slippery=False)
state, info = env.reset()
print("FrozenLake Simulation")
for step in range(20):
    action = random.randint(0, 3)
    next_state, reward, terminated, truncated, info = env.step(action)
    print("Step:", step + 1)
    print("State:", next_state)
    print("Action:", action)
    print("Reward:", reward)
    if terminated or truncated:
        if reward == 1:
            print("Goal Reached!")
        else:
```

```
print("Fell into a Hole!")
```

```
break
```

```
env.close()
```

Output:

```
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>
===== RESTART: /Users/vinnu/Documents/exp-1.py =====
FrozenLake Simulation
Step: 1
State: 0
Action: 3
Reward: 0
Step: 2
State: 4
Action: 1
Reward: 0
Step: 3
State: 0
Action: 3
Reward: 0
Step: 4
State: 0
Action: 3
Reward: 0
Step: 5
State: 0
Action: 3
Reward: 0
Step: 6
State: 4
Action: 1
Reward: 0
Step: 7
State: 5
Action: 2
Reward: 0
Fell into a Hole!
>> |
```

Ln: 35, Col: 0

Experiment 2:

Implement an ϵ -greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the performance for ϵ values of 0.1, 0.3, and 0.5 based on exploration, exploitation, and total rewards.

Program:

```
import random
reward = [4, 7, 10]
trials = 500
print("Epsilon-Greedy Multi-Armed Bandit")
for e in [0.1, 0.3, 0.5]:
    total = 0
    explore = 0
    exploit = 0
    for i in range(trials):
        x = random.random()
        if x < e:
            arm = random.randint(0, 2)
            explore += 1
        else:
            arm = 2
```

```
exploit += 1
```

```
total += reward[arm]
```

```
print("\nEpsilon =", e)
```

```
print("Exploration =", explore)
```

```
print("Exploitation =", exploit)
```

```
print("Total Reward =", total)
```

Output:

```
> python exp2.py
===== RESTART: /Users/vinnu/Documents/exp-2.py =====
Epsilon-Greedy Multi-Armed Bandit

Epsilon = 0.1
Exploration = 43
Exploitation = 457
Total Reward = 4886

Epsilon = 0.3
Exploration = 168
Exploitation = 332
Total Reward = 4514

Epsilon = 0.5
Exploration = 227
Exploitation = 273
Total Reward = 4310
>
```

Experiment 3:

Design a Markov Decision Process (MDP) for a robot moving from a start position to a goal. Define the states, actions, rewards, and transitions. Implement a simple Python program to simulate the MDP.

Program:

```
import random

states = ["Start", "Middle", "Goal"]
actions = ["Forward", "Stay"]
state = "Start"
reward = 0

print("MDP Robot Simulation")

while state != "Goal":
    action = random.choice(actions)
    print("Current State:", state)
    print("Action:", action)
    if state == "Start" and action == "Forward":
        state = "Middle"
        reward += 5
    elif state == "Middle" and action == "Forward":
        state = "Goal"
        reward += 10
    else:
```

```
reward -= 1
```

```
print("Final State:", state)
```

```
print("Total Reward:", reward)
```

Output:

```
>>
===== RESTART: /Users/vinnu/Documents/exp-3.py =====
MDP Robot Simulation
Current State: Start
Action: Forward
Current State: Middle
Action: Forward
Final State: Goal
Total Reward: 15
>>
```

Experiment 4:

Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.

Program:

```
import numpy as np

reward = [0, 0, -10, 0, 10]

value = np.zeros(5)

gamma = 0.9

print("Value Iteration")

for i in range(5):

    for s in range(4):

        if s == 2:

            continue

        value[s] = reward[s] + gamma * value[s + 1]

print("\nState Values")

for i in range(5):
```

```

    print("State", i, "=", round(value[i], 2))

print("\nPolicy")

for s in range(5):

    if s == 2:

        print("State", s, ": Restricted")

    elif s == 4:

        print("State", s, ": Goal")

    else:

        print("State", s, ": Move Right")

```

Output:

```

>
===== RESTART: /Users/vinnu/Documents/exp-4.py =====
Value Iteration

State Values
State 0 = 0.0
State 1 = 0.0
State 2 = 0.0
State 3 = 0.0
State 4 = 0.0

Policy
State 0 : Move Right
State 1 : Move Right
State 2 : Restricted
State 3 : Move Right
State 4 : Goal

```


Experiment 5:

A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

Program:

```
import random
```

```
battery = 30
```

```
deliveries = 0
```

```
while battery > 0:
```

```
    action = random.choice(["deliver", "move", "charge"])
```

```
    if action == "deliver" and battery >= 5:
```

```
        deliveries += 1
```

```
        battery -= 5
```

```
        reward = 10
```

```
elif action == "move":
```

```
    battery -= 2
```

```
    reward = -1
```

```
else:
```

```
    reward = 5
```

```
    break
```

```
print("Action:", action,
```

```
      "Battery:", battery,
```

```
      "Reward:", reward)
```

```
print("Total Deliveries:", deliveries)
```

```
print("Remaining Battery:", battery)
```

Output:

```
State 4 : Goal
```

```
>>>
```

```
===== RESTART: /Users/vinnu/Documents/exp-5.py =====
```

```
Action: move Battery: 28 Reward: -1
```

```
Action: deliver Battery: 23 Reward: 10
```

```
Action: deliver Battery: 18 Reward: 10
```

```
Action: deliver Battery: 13 Reward: 10
```

```
Total Deliveries: 3
```

```
Remaining Battery: 13
```

```
>>>
```